Open Investment

Testes automatizados de API com BDD (Gherkin / Cucumber)

Projeto Cidades ESG Inteligentes - Gerenciamento de Projetos Sociais

FIAP | Análise e Desenvolvimento de Sistemas | Turma 2TDSOC
Integrante: Lucas Gabriel Neres Sant'Ana - RM 565612

1. Objetivo e alinhamento ESG

Validar de forma automatizada as funcionalidades críticas da API Open Investment (Java 17, Spring Boot 3.2, Oracle XE), usando BDD. Os testes cobrem os 32 endpoints com validação de status code, corpo JSON e contrato por JSON Schema, em cenários positivos e negativos.

Pilar ESG	O que os testes garantem
Social	Cadastro de beneficiários e voluntários, registro de impacto social, início automático do projeto ao registrar o primeiro impacto, bloqueio de CPF duplicado.
Governança / Compliance	Bloqueio de alocação acima do orçamento, alerta ao atingir 80% da verba, controle de acesso por perfil (ADMIN, GESTOR), rejeição de token ausente ou inválido, trilha de auditoria de operações.

2. Estratégia e ferramentas

Item	Escolha
BDD	Cucumber 7 com Gherkin em português (66 cenários em 9 features, 438 passos)
Testes de API	REST Assured 5 (HTTP) + AssertJ
Contrato	JSON Schema (draft-07) validado com networknt json-schema-validator; 11 schemas
Execução	JUnit Platform Suite + Maven; local, Docker ou GitHub Actions
Dados	Cada cenário cria seus próprios dados via API (sem IDs fixos) e usa o seed apenas para as credenciais

Os cenários negativos cobrem: dados inválidos (400), recurso inexistente (404), regra de negócio violada (422), não autenticado (401) e sem permissão (403). As tags @positivo, @negativo, @contrato, @governanca e @compliance permitem executar grupos específicos.

3. Cobertura dos 32 endpoints

Endpoint	Feature	Cenários
POST /api/auth/login	01_autenticacao	login ADMIN e GESTOR, senha errada, e-mail inexistente, corpo vazio
GET /api/projetos	02_projetos, 09_contratos_api	listar, filtrar por status, contrato
GET /api/projetos/{id}	02_projetos	buscar por ID, 404
GET /api/projetos/orcamento-critico	03_alocacoes, 09_contratos_api	projeto com 85% da verba aparece na lista
POST /api/projetos	02_projetos	criar (ADMIN e GESTOR), 3 validações (400)
PUT /api/projetos/{id}	02_projetos	atualizar
PATCH /api/projetos/{id}/status	02_projetos	EM_ANDAMENTO (204), status inválido (422)
DELETE /api/projetos/{id}	02_projetos	ADMIN exclui (204), GESTOR bloqueado (403)
GET /api/alocacoes/projeto/{id}	03_alocacoes	listar por projeto
GET /api/alocacoes/{id}	03_alocacoes	buscar, 404
POST /api/alocacoes	03_alocacoes	dentro do orçamento (201), acima (422), 2 validações (400), projeto inexistente (404)
DELETE /api/alocacoes/{id}	03_alocacoes	excluir devolve saldo
GET /api/impactos/projeto/{id}	04_impactos	listar, projeto inexistente (404)
GET /api/impactos/projeto/{id}/contagem	04_impactos	contagem
POST /api/impactos	04_impactos	registrar (inicia o projeto), tipo inválido, beneficiário e projeto inexistentes
GET /api/beneficiarios	05_beneficiarios, 09_contratos_api	listar
GET /api/beneficiarios/{id}	05_beneficiarios	buscar, 404
POST /api/beneficiarios	05_beneficiarios	criar, CPF duplicado (422), nome em branco (400)
PUT /api/beneficiarios/{id}	05_beneficiarios	atualizar
DELETE /api/beneficiarios/{id}	05_beneficiarios	excluir (204) e confirmar 404
GET /api/voluntarios	06_voluntarios, 09_contratos_api	listar
GET /api/voluntarios/{id}	06_voluntarios	buscar, 404
POST /api/voluntarios	06_voluntarios	criar, CPF duplicado (422), e-mail inválido (400)
PUT /api/voluntarios/{id}	06_voluntarios	atualizar

Endpoint	Feature	Cenários
PATCH /api/voluntarios/{id}/inativar	06_voluntarios	inativar
POST /api/voluntarios/{id}/vincular-projeto/{idProjeto}	06_voluntarios, 08_auditoria_logs	vincular gera alerta e log; projeto inexistente (404)
GET /api/alertas/pendentes	07_alertas, 09_contratos_api	listar pendentes, sem token (401)
GET /api/alertas/projeto/{id}	07_alertas, 03_alocacoes	listar por projeto
PATCH /api/alertas/{id}/ler	07_alertas	marcar como lido, 404
PATCH /api/alertas/{id}/resolver	07_alertas	resolver, 404
GET /api/logs	08_auditoria_logs	ADMIN (200), GESTOR (403)
GET /api/logs/tabela/{tabela}	08_auditoria_logs	rastreabilidade de INSERT, sem token (401)

4. Cenários em Gherkin

Texto integral das features executadas (arquivos em src/test/resources/features).

01_autenticacao.feature

```
# language: pt
@autenticacao
Funcionalidade: Autenticação e controle de acesso
  Como usuário da plataforma Open Investment
  Quero me autenticar com e-mail e senha
  Para acessar apenas os recursos permitidos ao meu perfil (governança e segurança)

  @positivo @contrato
  Cenário: Login com credenciais válidas retorna token JWT
    Dado que não estou autenticado
    Quando envio "POST" para "/api/auth/login" com o corpo:
      """
      {"email": "admin@openinvestment.com", "senha": "Admin@123"}
      """
    Então o status da resposta é 200
    E o corpo respeita o contrato "auth"
    E o campo "tipo" é "Bearer"
    E o campo "perfil" é "ADMIN"

  @positivo
  Cenário: Login do perfil GESTOR retorna o perfil correto
    Dado que não estou autenticado
    Quando envio "POST" para "/api/auth/login" com o corpo:
      """
      {"email": "gestor@openinvestment.com", "senha": "Gestor@123"}
      """
    Então o status da resposta é 200
    E o corpo respeita o contrato "auth"
    E o campo "perfil" é "GESTOR"

  @negativo
  Cenário: Login com senha incorreta é recusado com 401
    Dado que não estou autenticado
    Quando envio "POST" para "/api/auth/login" com o corpo:
      """
      {"email": "admin@openinvestment.com", "senha": "senha-errada"}
      """
    Então o status da resposta é 401
    E o corpo respeita o contrato "erro"
    E o campo numérico "status" é igual a "401"

  @negativo
  Cenário: Login com e-mail inexistente é recusado com 401
    Dado que não estou autenticado
    Quando envio "POST" para "/api/auth/login" com o corpo:
      """
      {"email": "naoexiste@openinvestment.com", "senha": "Qualquer@123"}
      """
    Então o status da resposta é 401
    E o corpo respeita o contrato "erro"

  @negativo
  Cenário: Login sem preencher os campos retorna erro de validação
    Dado que não estou autenticado
    Quando envio "POST" para "/api/auth/login" com o corpo:
      """
      {}
      """
    Então o status da resposta é 400
    E o corpo respeita o contrato "erro-validacao"
    E o campo "campos.email" existe
    E o campo "campos.senha" existe

  @negativo @governanca
  Cenário: Rota protegida sem token retorna 401
    Dado que não estou autenticado
    Quando envio "GET" para "/api/projetos"
    Então o status da resposta é 401
    E o corpo respeita o contrato "erro"

  @negativo @governanca
```

```
  Cenário: Token JWT malformado é rejeitado com 401
    Dado que uso o token inválido "abc.def.ghi"
    Quando envio "GET" para "/api/projetos"
    Então o status da resposta é 401
    E o corpo respeita o contrato "erro"
```

02_projetos.feature

```
# language: pt
@projetos
Funcionalidade: Gestão de projetos sociais
  Como gestor de projetos sociais ESG
  Quero cadastrar, consultar, atualizar e encerrar projetos
  Para acompanhar o investimento social com rastreabilidade

  Contexto:
    Dado que estou autenticado como "ADMIN"

  @positivo @contrato
  Cenário: Listar projetos cadastrados
    Quando envio "GET" para "/api/projetos"
    Então o status da resposta é 200
    E o corpo é uma lista
    E cada item da lista respeita o contrato "projeto"

  @positivo @contrato
  Cenário: Criar projeto com dados válidos
    Quando envio "POST" para "/api/projetos" com o corpo:
      """
      {"nomeProjeto": "Projeto BDD {{unique}}", "descricao": "Horta comunitaria", "odsVinculacao": "ODS 2 -
Fome Zero", "dataInicio": "2026-03-01", "dataFimPrev": "2026-12-31", "orcamentoTotal": 1000.00}
      """
    Então o status da resposta é 201
    E o corpo respeita o contrato "projeto"
    E o campo "statusProjeto" é "PLANEJADO"
    E o campo numérico "verbaUtilizada" é igual a "0"
    E o campo numérico "saldoDisponivel" é igual a "1000"

  @negativo
  Esquema do Cenário: Rejeitar projeto com dados inválidos (<motivo>)
    Quando envio "POST" para "/api/projetos" com o corpo:
      """
      {"nomeProjeto": "<nome>", "dataInicio": "2026-03-01", "orcamentoTotal": <orcamento>}
      """
    Então o status da resposta é 400
    E o corpo respeita o contrato "erro-validacao"
    E o campo "campos.<campo>" existe

    Exemplos:
      | motivo             | nome      | orcamento | campo          |
      | nome em branco     |           | 1000.00   | nomeProjeto    |
      | orçamento zero     | Projeto X | 0         | orcamentoTotal |
      | orçamento negativo | Projeto X | -50.00    | orcamentoTotal |

  @positivo @contrato
  Cenário: Buscar projeto por ID
    Dado que existe um projeto com orçamento de 1000 reais
    Quando envio "GET" para "/api/projetos/{{projetoId}}"
    Então o status da resposta é 200
    E o corpo respeita o contrato "projeto"
    E o campo "idProjeto" é "{{projetoId}}"

  @negativo
  Cenário: Buscar projeto inexistente retorna 404
    Quando envio "GET" para "/api/projetos/999999999"
    Então o status da resposta é 404
    E o corpo respeita o contrato "erro"
    E o campo numérico "status" é igual a "404"

  @positivo
  Cenário: Filtrar projetos por status
    Dado que existe um projeto com orçamento de 1000 reais
    Quando envio "GET" para "/api/projetos?status=PLANEJADO"
    Então o status da resposta é 200
    E cada item da lista respeita o contrato "projeto"
    E a lista contém um item com "idProjeto" igual a "{{projetoId}}"

  @positivo
  Cenário: Atualizar dados de um projeto
```

```
    Dado que existe um projeto com orçamento de 1000 reais
    Quando envio "PUT" para "/api/projetos/{{projetoId}}" com o corpo:
      """
      {"nomeProjeto": "Projeto BDD Atualizado {{unique}}", "descricao": "Descricao revisada", "odsVinculacao":
 "ODS 4", "dataInicio": "2026-04-01", "dataFimPrev": "2027-03-31", "orcamentoTotal": 2000.00}
      """
    Então o status da resposta é 200
    E o corpo respeita o contrato "projeto"
    E o campo "nomeProjeto" é "Projeto BDD Atualizado {{unique}}"
    E o campo numérico "orcamentoTotal" é igual a "2000"

  @positivo
  Cenário: Alterar o status do projeto para EM_ANDAMENTO
    Dado que existe um projeto com orçamento de 1000 reais
    Quando envio "PATCH" para "/api/projetos/{{projetoId}}/status?status=EM_ANDAMENTO"
    Então o status da resposta é 204
    Quando envio "GET" para "/api/projetos/{{projetoId}}"
    Então o status da resposta é 200
    E o campo "statusProjeto" é "EM_ANDAMENTO"

  @negativo
  Cenário: Status inexistente viola regra de negócio (422)
    Dado que existe um projeto com orçamento de 1000 reais
    Quando envio "PATCH" para "/api/projetos/{{projetoId}}/status?status=INEXISTENTE"
    Então o status da resposta é 422
    E o corpo respeita o contrato "erro"
    E o campo numérico "status" é igual a "422"

  @positivo
  Cenário: ADMIN exclui projeto sem registros dependentes
    Dado que existe um projeto com orçamento de 1000 reais
    Quando envio "DELETE" para "/api/projetos/{{projetoId}}"
    Então o status da resposta é 204
    Quando envio "GET" para "/api/projetos/{{projetoId}}"
    Então o status da resposta é 404

  @negativo @governanca
  Cenário: GESTOR não tem permissão para excluir projeto
    Dado que existe um projeto com orçamento de 1000 reais
    E que estou autenticado como "GESTOR"
    Quando envio "DELETE" para "/api/projetos/{{projetoId}}"
    Então o status da resposta é 403
    E o corpo respeita o contrato "erro"

  @positivo @governanca
  Cenário: GESTOR pode criar projeto
    Dado que estou autenticado como "GESTOR"
    Quando envio "POST" para "/api/projetos" com o corpo:
      """
      {"nomeProjeto": "Projeto Gestor {{unique}}", "dataInicio": "2026-03-01", "orcamentoTotal": 500.00}
      """
    Então o status da resposta é 201
    E o corpo respeita o contrato "projeto"
```

03_alocacoes.feature

```
# language: pt
@alocacoes @compliance
Funcionalidade: Alocação de recursos e controle orçamentário
  Como gestor financeiro
  Quero alocar recursos respeitando o orçamento do projeto
  Para garantir compliance e evitar estouro de verba

  Contexto:
    Dado que estou autenticado como "ADMIN"
    E que existe um projeto com orçamento de 1000 reais

  @positivo @contrato
  Cenário: Alocar valor dentro do orçamento
    Quando envio "POST" para "/api/alocacoes" com o corpo:
      """
      {"idProjeto": {{projetoId}}, "descricao": "Compra de equipamentos", "valor": 400.00, "tipoRecurso":
"FINANCEIRO"}
      """
    Então o status da resposta é 201
    E o corpo respeita o contrato "alocacao"
    E o campo "tipoRecurso" é "FINANCEIRO"
    E o campo numérico "valor" é igual a "400"
  @negativo @compliance
```

```
  Cenário: Alocação acima do orçamento é bloqueada com 422
    Quando envio "POST" para "/api/alocacoes" com o corpo:
      """
      {"idProjeto": {{projetoId}}, "descricao": "Valor acima do orçamento", "valor": 1500.00, "tipoRecurso":
"FINANCEIRO"}
      """
    Então o status da resposta é 422
    E o corpo respeita o contrato "erro"
    E o campo "message" contém "Saldo restante"

  @negativo
  Esquema do Cenário: Rejeitar alocação com dados inválidos (<motivo>)
    Quando envio "POST" para "/api/alocacoes" com o corpo:
      """
      {"idProjeto": {{projetoId}}, "valor": <valor>, "tipoRecurso": "<tipo>"}
      """
    Então o status da resposta é 400
    E o corpo respeita o contrato "erro-validacao"
    E o campo "campos.<campo>" existe

    Exemplos:
      | motivo           | valor  | tipo       | campo       |
      | tipo desconhecido | 100.00 | COMBUSTIVEL | tipoRecurso |
      | valor zero       | 0      | FINANCEIRO | valor       |

  @negativo
  Cenário: Alocar em projeto inexistente retorna 404
    Quando envio "POST" para "/api/alocacoes" com o corpo:
      """
      {"idProjeto": 999999999, "valor": 10.00, "tipoRecurso": "FINANCEIRO"}
      """
    Então o status da resposta é 404
    E o corpo respeita o contrato "erro"

  @positivo @contrato
  Cenário: Listar alocações de um projeto
    Dado que existe uma alocação de 300 reais neste projeto
    Quando envio "GET" para "/api/alocacoes/projeto/{{projetoId}}"
    Então o status da resposta é 200
    E o corpo é uma lista
    E cada item da lista respeita o contrato "alocacao"
    E a lista contém um item com "idAlocacao" igual a "{{alocacaoId}}"

  @positivo @contrato
  Cenário: Buscar alocação por ID
    Dado que existe uma alocação de 300 reais neste projeto
    Quando envio "GET" para "/api/alocacoes/{{alocacaoId}}"
    Então o status da resposta é 200
    E o corpo respeita o contrato "alocacao"

  @negativo
  Cenário: Buscar alocação inexistente retorna 404
    Quando envio "GET" para "/api/alocacoes/999999999"
    Então o status da resposta é 404
    E o corpo respeita o contrato "erro"

  @positivo
  Cenário: Excluir alocação devolve o saldo ao projeto
    Dado que existe uma alocação de 300 reais neste projeto
    Quando envio "DELETE" para "/api/alocacoes/{{alocacaoId}}"
    Então o status da resposta é 204
    Quando envio "GET" para "/api/alocacoes/{{alocacaoId}}"
    Então o status da resposta é 404
    Quando envio "GET" para "/api/projetos/{{projetoId}}"
    Então o campo numérico "verbaUtilizada" é igual a "0"

  @positivo @compliance @governanca
  Cenário: Uso de 80% do orçamento gera alerta e marca o projeto como crítico
    Dado que existe uma alocação de 850 reais neste projeto
    Quando envio "GET" para "/api/alertas/projeto/{{projetoId}}"
    Então o status da resposta é 200
    E cada item da lista respeita o contrato "alerta"
    E a lista contém um item com "tipoAlerta" igual a "ORCAMENTO_CRITICO"
    Quando envio "GET" para "/api/projetos/orcamento-critico"
    Então o status da resposta é 200
    E cada item da lista respeita o contrato "projeto"
    E a lista contém um item com "idProjeto" igual a "{{projetoId}}"
```

04_impactos.feature

```
# language: pt
@impactos @social
Funcionalidade: Registro de impacto social
  Como responsável pelo pilar Social do ESG
  Quero registrar os beneficiários atendidos por cada projeto
  Para medir e comprovar o impacto social gerado

  Contexto:
    Dado que estou autenticado como "ADMIN"
    E que existe um projeto com orçamento de 1000 reais
    E que existe um beneficiário cadastrado

  @positivo @contrato
  Cenário: Registrar impacto inicia automaticamente o projeto
    Quando envio "POST" para "/api/impactos" com o corpo:
      """
      {"idProjeto": {{projetoId}}, "idBeneficiario": {{beneficiarioId}}, "tipoImpacto": "CAPACITACAO",
"descricao": "Curso de informatica"}
      """
    Então o status da resposta é 201
    E o corpo respeita o contrato "impacto"
    E o campo "tipoImpacto" é "CAPACITACAO"
    Quando envio "GET" para "/api/projetos/{{projetoId}}"
    Então o campo "statusProjeto" é "EM_ANDAMENTO"
    Quando envio "GET" para "/api/alertas/projeto/{{projetoId}}"
    Então a lista contém um item com "tipoAlerta" igual a "PROJETO_INICIADO"

  @negativo
  Cenário: Tipo de impacto inválido retorna erro de validação
    Quando envio "POST" para "/api/impactos" com o corpo:
      """
      {"idProjeto": {{projetoId}}, "idBeneficiario": {{beneficiarioId}}, "tipoImpacto": "INVALIDO"}
      """
    Então o status da resposta é 400
    E o corpo respeita o contrato "erro-validacao"
    E o campo "campos.tipoImpacto" existe

  @negativo
  Cenário: Beneficiário inexistente retorna 404
    Quando envio "POST" para "/api/impactos" com o corpo:
      """
      {"idProjeto": {{projetoId}}, "idBeneficiario": 999999999, "tipoImpacto": "DOACAO"}
      """
    Então o status da resposta é 404
    E o corpo respeita o contrato "erro"

  @negativo
  Cenário: Projeto inexistente retorna 404
    Quando envio "POST" para "/api/impactos" com o corpo:
      """
      {"idProjeto": 999999999, "idBeneficiario": {{beneficiarioId}}, "tipoImpacto": "DOACAO"}
      """
    Então o status da resposta é 404
    E o corpo respeita o contrato "erro"

  @positivo @contrato
  Cenário: Listar impactos de um projeto
    Dado que existe um impacto registrado neste projeto
    Quando envio "GET" para "/api/impactos/projeto/{{projetoId}}"
    Então o status da resposta é 200
    E o corpo é uma lista
    E cada item da lista respeita o contrato "impacto"
    E a lista contém um item com "idImpacto" igual a "{{impactoId}}"

  @positivo @contrato
  Cenário: Contar impactos de um projeto
    Dado que existe um impacto registrado neste projeto
    Quando envio "GET" para "/api/impactos/projeto/{{projetoId}}/contagem"
    Então o status da resposta é 200
    E o corpo respeita o contrato "contagem"
    E o campo numérico "totalImpactos" é igual a "1"

  @negativo
  Cenário: Listar impactos de projeto inexistente retorna 404
    Quando envio "GET" para "/api/impactos/projeto/999999999"
    Então o status da resposta é 404
    E o corpo respeita o contrato "erro"
```

05_beneficiarios.feature

```
# language: pt
@beneficiarios @social
Funcionalidade: Cadastro de beneficiários
  Como responsável pelo pilar Social do ESG
  Quero manter o cadastro de pessoas atendidas
  Para garantir inclusão e evitar registros duplicados

  Contexto:
    Dado que estou autenticado como "GESTOR"

  @positivo @contrato
  Cenário: Listar beneficiários
    Quando envio "GET" para "/api/beneficiarios"
    Então o status da resposta é 200
    E o corpo é uma lista
    E cada item da lista respeita o contrato "beneficiario"

  @positivo @contrato
  Cenário: Cadastrar beneficiário com dados válidos
    Quando envio "POST" para "/api/beneficiarios" com o corpo:
      """
      {"nome": "Beneficiario BDD {{unique}}", "cpf": "{{cpf}}", "perfil": "Jovem 18-25", "regiao": "Zona
Leste - SP"}
      """
    Então o status da resposta é 201
    E o corpo respeita o contrato "beneficiario"
    E o campo "cpf" é "{{cpf}}"

  @negativo @governanca
  Cenário: CPF duplicado é recusado com 422
    Dado que existe um beneficiário cadastrado
    Quando envio "POST" para "/api/beneficiarios" com o corpo:
      """
      {"nome": "Outro Beneficiario", "cpf": "{{cpf}}"}
      """
    Então o status da resposta é 422
    E o corpo respeita o contrato "erro"
    E o campo "message" contém "CPF"

  @negativo
  Cenário: Cadastro sem nome retorna erro de validação
    Quando envio "POST" para "/api/beneficiarios" com o corpo:
      """
      {"nome": "", "cpf": "{{cpf}}"}
      """
    Então o status da resposta é 400
    E o corpo respeita o contrato "erro-validacao"
    E o campo "campos.nome" existe

  @positivo @contrato
  Cenário: Buscar beneficiário por ID
    Dado que existe um beneficiário cadastrado
    Quando envio "GET" para "/api/beneficiarios/{{beneficiarioId}}"
    Então o status da resposta é 200
    E o corpo respeita o contrato "beneficiario"
    E o campo "idBeneficiario" é "{{beneficiarioId}}"

  @negativo
  Cenário: Buscar beneficiário inexistente retorna 404
    Quando envio "GET" para "/api/beneficiarios/999999999"
    Então o status da resposta é 404
    E o corpo respeita o contrato "erro"

  @positivo
  Cenário: Atualizar beneficiário
    Dado que existe um beneficiário cadastrado
    Quando envio "PUT" para "/api/beneficiarios/{{beneficiarioId}}" com o corpo:
      """
      {"nome": "Beneficiario Atualizado {{unique}}", "cpf": "{{cpf}}", "perfil": "Adulto 26-45", "regiao":
"Zona Sul - SP"}
      """
    Então o status da resposta é 200
    E o corpo respeita o contrato "beneficiario"
    E o campo "regiao" é "Zona Sul - SP"

  @positivo
  Cenário: Excluir beneficiário sem impactos vinculados
    Dado que existe um beneficiário cadastrado
    Quando envio "DELETE" para "/api/beneficiarios/{{beneficiarioId}}"
    Então o status da resposta é 204
    Quando envio "GET" para "/api/beneficiarios/{{beneficiarioId}}"
```

```
    Então o status da resposta é 404
```

06_voluntarios.feature

```
# language: pt
@voluntarios @social
Funcionalidade: Gestão de voluntários
  Como coordenador de voluntariado
  Quero cadastrar voluntários e vinculá-los a projetos
  Para ampliar o impacto social com rastreabilidade

  Contexto:
    Dado que estou autenticado como "ADMIN"

  @positivo @contrato
  Cenário: Listar voluntários
    Quando envio "GET" para "/api/voluntarios"
    Então o status da resposta é 200
    E o corpo é uma lista
    E cada item da lista respeita o contrato "voluntario"

  @positivo @contrato
  Cenário: Cadastrar voluntário com dados válidos
    Quando envio "POST" para "/api/voluntarios" com o corpo:
      """
      {"nome": "Voluntario BDD {{unique}}", "cpf": "{{cpf}}", "email": "{{email}}", "especialidade":
"Pedagogia"}
      """
    Então o status da resposta é 201
    E o corpo respeita o contrato "voluntario"
    E o campo "statusVol" é "ATIVO"

  @negativo @governanca
  Cenário: CPF duplicado é recusado com 422
    Dado que existe um voluntário cadastrado
    Quando envio "POST" para "/api/voluntarios" com o corpo:
      """
      {"nome": "Outro Voluntario", "cpf": "{{cpf}}", "email": "outro{{email}}"}
      """
    Então o status da resposta é 422
    E o corpo respeita o contrato "erro"

  @negativo
  Cenário: E-mail inválido retorna erro de validação
    Quando envio "POST" para "/api/voluntarios" com o corpo:
      """
      {"nome": "Voluntario Invalido", "cpf": "{{cpf}}", "email": "email-sem-arroba"}
      """
    Então o status da resposta é 400
    E o corpo respeita o contrato "erro-validacao"
    E o campo "campos.email" existe

  @positivo @contrato
  Cenário: Buscar voluntário por ID
    Dado que existe um voluntário cadastrado
    Quando envio "GET" para "/api/voluntarios/{{voluntarioId}}"
    Então o status da resposta é 200
    E o corpo respeita o contrato "voluntario"
    E o campo "idVoluntario" é "{{voluntarioId}}"

  @negativo
  Cenário: Buscar voluntário inexistente retorna 404
    Quando envio "GET" para "/api/voluntarios/999999999"
    Então o status da resposta é 404
    E o corpo respeita o contrato "erro"

  @positivo
  Cenário: Atualizar voluntário
    Dado que existe um voluntário cadastrado
    Quando envio "PUT" para "/api/voluntarios/{{voluntarioId}}" com o corpo:
      """
      {"nome": "Voluntario Atualizado {{unique}}", "cpf": "{{cpf}}", "email": "{{email}}", "especialidade":
"Engenharia"}
      """
    Então o status da resposta é 200
    E o corpo respeita o contrato "voluntario"
    E o campo "especialidade" é "Engenharia"

  @positivo
```

```
  Cenário: Inativar voluntário
    Dado que existe um voluntário cadastrado
    Quando envio "PATCH" para "/api/voluntarios/{{voluntarioId}}/inativar"
    Então o status da resposta é 204
    Quando envio "GET" para "/api/voluntarios/{{voluntarioId}}"
    Então o campo "statusVol" é "INATIVO"

  @positivo @governanca
  Cenário: Vincular voluntário a um projeto gera alerta
    Dado que existe um voluntário cadastrado
    E que existe um projeto com orçamento de 1000 reais
    Quando envio "POST" para "/api/voluntarios/{{voluntarioId}}/vincular-projeto/{{projetoId}}"
    Então o status da resposta é 204
    Quando envio "GET" para "/api/alertas/projeto/{{projetoId}}"
    Então a lista contém um item com "tipoAlerta" igual a "NOVO_VOLUNTARIO"

  @negativo
  Cenário: Vincular voluntário a projeto inexistente retorna 404
    Dado que existe um voluntário cadastrado
    Quando envio "POST" para "/api/voluntarios/{{voluntarioId}}/vincular-projeto/999999999"
    Então o status da resposta é 404
    E o corpo respeita o contrato "erro"
```

07_alertas.feature

```
# language: pt
@alertas @governanca
Funcionalidade: Alertas ESG
  Como gestor de projetos
  Quero acompanhar e tratar alertas de orçamento e voluntariado
  Para agir rapidamente sobre riscos de governança

  Contexto:
    Dado que estou autenticado como "GESTOR"
    E que existe um projeto com orçamento de 1000 reais
    E que existe uma alocação de 900 reais neste projeto

  @positivo @contrato
  Cenário: Listar alertas pendentes
    Quando envio "GET" para "/api/alertas/pendentes"
    Então o status da resposta é 200
    E o corpo é uma lista
    E cada item da lista respeita o contrato "alerta"
    E a lista contém um item com "tipoAlerta" igual a "ORCAMENTO_CRITICO"

  @positivo @contrato
  Cenário: Listar alertas de um projeto
    Quando envio "GET" para "/api/alertas/projeto/{{projetoId}}"
    Então o status da resposta é 200
    E cada item da lista respeita o contrato "alerta"
    E a lista contém um item com "statusAlerta" igual a "PENDENTE"

  @positivo
  Cenário: Marcar alerta como lido
    Quando envio "GET" para "/api/alertas/projeto/{{projetoId}}"
    E guardo o campo "0.idAlerta" da resposta como "alertaId"
    E envio "PATCH" para "/api/alertas/{{alertaId}}/ler"
    Então o status da resposta é 200
    E o corpo respeita o contrato "alerta"
    E o campo "statusAlerta" é "LIDO"

  @positivo
  Cenário: Resolver alerta
    Quando envio "GET" para "/api/alertas/projeto/{{projetoId}}"
    E guardo o campo "0.idAlerta" da resposta como "alertaId"
    E envio "PATCH" para "/api/alertas/{{alertaId}}/resolver"
    Então o status da resposta é 200
    E o corpo respeita o contrato "alerta"
    E o campo "statusAlerta" é "RESOLVIDO"

  @negativo
  Cenário: Marcar alerta inexistente como lido retorna 404
    Quando envio "PATCH" para "/api/alertas/999999999/ler"
    Então o status da resposta é 404
    E o corpo respeita o contrato "erro"

  @negativo
  Cenário: Resolver alerta inexistente retorna 404
    Quando envio "PATCH" para "/api/alertas/999999999/resolver"
```

```
    Então o status da resposta é 404
    E o corpo respeita o contrato "erro"

  @negativo @governanca
  Cenário: Alertas sem autenticação retornam 401
    Dado que não estou autenticado
    Quando envio "GET" para "/api/alertas/pendentes"
    Então o status da resposta é 401
    E o corpo respeita o contrato "erro"
```

08_auditoria_logs.feature

```
# language: pt
@auditoria @governanca
Funcionalidade: Trilha de auditoria
  Como auditor (perfil ADMIN)
  Quero consultar o log de operações do sistema
  Para garantir integridade, rastreabilidade e conformidade

  @positivo @contrato
  Cenário: ADMIN consulta o log de auditoria paginado
    Dado que estou autenticado como "ADMIN"
    Quando envio "GET" para "/api/logs?page=0&size=5"
    Então o status da resposta é 200
    E o corpo respeita o contrato "pagina-logs"

  @positivo @contrato
  Cenário: Criar projeto registra operação INSERT na auditoria
    Dado que estou autenticado como "ADMIN"
    E que existe um projeto com orçamento de 1000 reais
    Quando envio "GET" para "/api/logs/tabela/projeto_social"
    Então o status da resposta é 200
    E o corpo respeita o contrato "pagina-logs"
    E a lista em "content" contém um item com "idReferencia" igual a "{{projetoId}}"
    E a lista em "content" contém um item com "operacao" igual a "INSERT"

  @positivo
  Cenário: Vincular voluntário a projeto fica registrado na auditoria
    Dado que estou autenticado como "ADMIN"
    E que existe um voluntário cadastrado
    E que existe um projeto com orçamento de 1000 reais
    Quando envio "POST" para "/api/voluntarios/{{voluntarioId}}/vincular-projeto/{{projetoId}}"
    Então o status da resposta é 204
    Quando envio "GET" para "/api/logs/tabela/voluntario_projeto"
    Então o status da resposta é 200
    E a lista em "content" contém um item com "idReferencia" igual a "{{projetoId}}"

  @negativo
  Cenário: GESTOR não pode consultar o log de auditoria
    Dado que estou autenticado como "GESTOR"
    Quando envio "GET" para "/api/logs"
    Então o status da resposta é 403
    E o corpo respeita o contrato "erro"

  @negativo
  Cenário: Log de auditoria sem autenticação retorna 401
    Dado que não estou autenticado
    Quando envio "GET" para "/api/logs/tabela/projeto_social"
    Então o status da resposta é 401
    E o corpo respeita o contrato "erro"
```

09_contratos_api.feature

```
# language: pt
@contratos
Funcionalidade: Contratos das APIs de consulta (JSON Schema)
  Como consumidor da API
  Quero que as listagens respeitem um contrato estável
  Para integrar sistemas sem quebras inesperadas

  @contrato @positivo
  Esquema do Cenário: Listagem <endpoint> respeita o contrato <contrato>
    Dado que estou autenticado como "ADMIN"
    Quando envio "GET" para "<endpoint>"
    Então o status da resposta é 200
    E o corpo é uma lista
    E cada item da lista respeita o contrato "<contrato>"
```

Exemplos:

endpoint	contrato
/api/projetos	projeto
/api/projetos/orcamento-critico	projeto
/api/beneficiarios	beneficiario
/api/voluntarios	voluntario
/api/alertas/pendentes	alerta

5. Testes de contrato (JSON Schema)

Cada resposta de sucesso e de erro é validada contra um schema em src/test/resources/schemas: alerta, alocacao, auth, beneficiario, contagem, erro-validacao, erro, impacto, pagina-logs, projeto, voluntario. Exemplo (projeto.schema.json):

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "Projeto social",
  "type": "object",
  "required": [
    "idProjeto",
    "nomeProjeto",
    "orcamentoTotal",
    "verbaUtilizada",
    "saldoDisponivel",
    "statusProjeto"
  ],
  "properties": {
    "idProjeto": {
      "type": "integer"
    },
    "nomeProjeto": {
      "type": "string",
      "minLength": 1
    },
    "descricao": {
      "type": [
        "string",
        "null"
      ]
    },
    "odsVinculacao": {
      "type": [
        "string",
        "null"
      ]
    },
    "dataInicio": {
      "type": [
        "string",
        "null"
      ]
    },
    "dataFimPrev": {
      "type": [
        "string",
        "null"
      ]
    },
    "orcamentoTotal": {
      "type": "number"
    },
    "verbaUtilizada": {
      "type": "number"
    },
    "saldoDisponivel": {
      "type": "number"
    },
    "percentualUtilizado": {
      "type": [
        "number",
        "null"
      ]
    },
    "statusProjeto": {
      "type": "string",
      "enum": [
        "PLANEJADO",
        "EM_ANDAMENTO",
        "CONCLUIDO",
        "CANCELADO"
      ]
    },
    "criadoEm": {
      "type": [
        "string",
        "null"
      ]
```

```
    }
  }
}
```

6. Configuração e execução

junit-platform.properties

```
cucumber.plugin=pretty, summary, html:target/cucumber-report/cucumber.html, json:target/cucumber-report/
cucumber.json, junit:target/cucumber-report/cucumber.xml
cucumber.publish.quiet=true
```

Dockerfile

```
# Executa os testes BDD contra uma API já em execução.
# Build : docker build -t open-investment-bdd .
# Run   : docker run --rm -e BASE_URL=http://host.docker.internal:8080 open-investment-bdd
FROM maven:3.9.6-eclipse-temurin-17
WORKDIR /bdd
COPY pom.xml .
COPY src ./src
ENV BASE_URL=http://host.docker.internal:8080
CMD ["mvn", "-B", "-ntp", "test"]
```

Passo do pipeline (.github/workflows/ci.yml, job deploy-staging)

```
- name: Testes BDD de API (Cucumber + REST Assured + JSON Schema)
  run: mvn -B -ntp -f bdd-tests/pom.xml test | tee bdd.log
  env:
    BASE_URL: http://localhost:8081
```

Execução local

```
docker compose up -d --build --wait      # na raiz do projeto: API em http://localhost:8080
cd bdd-tests && mvn test                   # relatorio: target/cucumber-report/cucumber.html
mvn test -Dcucumber.filter.tags="@negativo"  # somente cenarios negativos
docker build -t open-investment-bdd . && docker run --rm -e BASE_URL=http://host.docker.internal:8080 open-
investment-bdd
```

Dependências principais (pom.xml)

```
io.cucumber:cucumber-java, cucumber-junit-platform-engine, cucumber-picocontainer (7.15.0)
org.junit.platform:junit-platform-suite (5.10.2)
io.rest-assured:rest-assured (5.4.0)
com.networknt:json-schema-validator (1.4.0)
com.fasterxml.jackson.core:jackson-databind (2.17.0)
org.assertj:assertj-core (3.25.3)
```

7. Evidências de execução

Execução local: console do mvn test com o resumo de cenários e passos

PRINT PENDENTE: adicione docs/prints/01-execucao-local-console.png e rode docs/gerar-pdf.py novamente.

Relatório HTML do Cucumber (target/cucumber-report/cucumber.html)

PRINT PENDENTE: adicione docs/prints/02-relatorio-html-cucumber.png e rode docs/gerar-pdf.py novamente.

GitHub Actions: job Deploy - Staging com o passo de testes BDD verde

PRINT PENDENTE: adicione docs/prints/03-pipeline-bdd-actions.png e rode docs/gerar-pdf.py novamente.

GitHub Actions: resumo do job com o total de cenários executados

PRINT PENDENTE: adicione docs/prints/04-resumo-bdd-no-job.png e rode docs/gerar-pdf.py novamente.

8. Defeito encontrado e corrigido

Ao escrever o cenário negativo de login (senha incorreta), a análise do código mostrou que a API respondia HTTP 500: a exceção de autenticação era capturada pelo handler genérico de exceções. Correção: novo handler para AuthenticationException em GlobalExceptionHandler, que devolve 401 com o corpo padrão de erro. O cenário passou a validar o comportamento correto (401 e contrato 'erro').

9. Checklist

Item	OK
Mínimo de 3 cenários BDD em Gherkin (entregues: 66, positivos e negativos)	[x]
Testes de API para todas as APIs (32 endpoints)	[x]
Validação de status code e corpo JSON	[x]
Testes de contrato com JSON Schema	[x]
Execução documentada (local, Docker e pipeline)	[x]
Código-fonte, Dockerfile e README no .zip	[x]
Evidências de execução (prints) neste PDF	[ ]

O item de evidências depende dos prints em docs/prints/. Rode docs/gerar-pdf.py depois de adicioná-los.